災害時の見回りを自律で行う四足ロボット

強化学習 (歩行・起き上がり) × 経路計画 × 画像言語モデルの分業

Physical AI 応用 1 講座 最終課題 説明資料 (2026-10)

1. 課題の概要と目的

災害直後の屋外 (倒壊した壁、瓦礫、崩れた段差や階段) を、四足ロボットが **人の指示は最初の一回だけで** 見回り、要救助者らしきものを見つけたら人に確認して位置を通報し、余震で転んでも自力で起き上がって任務を続ける。 通信が細く人手も足りない災害現場では「指示待ち」のロボットは役に立たないので、**自律性** と **失敗からの回復** を設計の中心に置いた。 シミュレータは Genesis (GPU 並列の物理)、ロボットは Unitree Go2 (12 関節)。歩行と起き上がりは強化学習、経路は既知の地図の A*、未知の段差は前方の高さスキャナ、認識はローカルの画像言語モデル (VLM)、人との対話は YES/NO。

2. システム構成

層	中身
シミュレータ	Genesis 1.4 (DART 物理、4,096 体を GPU で並列)。現場は 18 m 四方 (3×3 の区画: 砂利、箱の散らばり、坂、ピラミッド階段、中央は出発点) に、倒れた壁 ×2 (0.5 m)、瓦礫の山 (0.4 m)、倒れた人形 (マネキン) を配置
歩行 (RL)	PPO、観測 = 胴体角速度・重力方向・速度指令・関節角と角速度・前回出力 (45 次元)、出力 = 12 関節の目標角 (PD 制御)。現場の地形の上で、速度 0.2〜0.8 m/s・旋回 ±0.8 rad/s・3 秒ごとの横からの衝撃 (最大 1.0 m/s) を混ぜて 1,000 反復 (6 分)
起き上がり (RL)	横倒し・仰向け・関節ばらばらの姿勢から始め、「上向き度 (仰向け 0 → 立ち 1 の連続値) と立ち高さ」を報酬に 600 反復 (5 分)。現場の地形の上で学習した版を任

	務で使用
経路計画	既知の障害物 (ドローン空撮の地図に相当) を 0.8 m 膨らませた 0.25 m 格子で A*。GPS の推定位置から 5 秒ごとに再計画し、1.3 m 先の点へ向かう
高さスキャナ	進行方向の前方 0.4〜1.8 m、幅 ±0.35 m の 15 点の地形高さ (LiDAR の見立て)。一段で 18 cm 超の段差か 60 cm 超の高さがあれば、空いている向き (35/70/105/150/180°) へ事前に迂回。地図に載っている障害物は見ない (経路計画と分担)
GPS	1 秒に 1 回、真の位置に ±1.5 m のばらつきを乗せ、直近 3 回の平均を使う。到着判定は 1.2 m
認識 (VLM)	経由地に着くと頭のカメラで 8 方向を撮り、Qwen3-VL-8B (ローカル、1 枚 0.5〜1 秒) に「人の形をした物体は写っていますか」と聞く。4 m 超の遠方なら記録だけして近い経由地で再確認
司令役	約 300 行の規則: 転倒 (0.3 秒) → 起き上がり → 0.5 秒立てたら歩行、8 秒立てなければ体勢を変える / 4 秒進めなければ迂回 / 8 秒動けなければ押し出し、それでも駄目なら起き上がり方策で体勢を崩す / 真後ろを向くときは回る向きを固定 / 発見 → 人に YES/NO → 近づいて通報 → 来た道を戻る



図 1: 見回り中 (右上は俯瞰、字幕は司令役の記録から自動生成)

3. 工夫した点

3.1 練習は本番より厳しく (外乱と地形のカリキュラム)

平地で育てた歩行方策は 1.0 m/s の横からの衝撃で転び (7 回中 2 回)、学習中にランダムな衝撃を混ぜた方策は転ばない (0 回)。耐えられる強さは「学習で経験した強さまで」で、2.0 m/s で育てると 2.0 まで無傷、2.5 で崖のように転ぶ (表 1)。地形も同じで、階段育ちは 4〜11 cm の段を 8 段登るが平地育ちは 1 段目で止まる。この「見たことのある意地悪には強い」という性質が、現場の地形と衝撃を混ぜて学習する設計の根拠になった。

蹴り (m/s)	平地育ち	蹴り 1.0 で育った	蹴り 3.0 で育った
1.0	転倒 1 /	0 / 8.5 m	0 / 8.1 m

	0.9 m		
2.0	6 / 0.3 m	3 / 1.2 m	**0 / 7.6 m**
2.5	7 / 0.3 m	6 / 0.2 m	2 / 1.1 m
3.0	7 / 0.3 m	7 / 0.3 m	5 / 0.8 m

表 1: 蹴り耐性 (2 秒ごとに 7 回の蹴り。転倒回数 / 15 秒で進んだ距離、同じ乱数列)

3.2 反射は規則、認識と対話は VLM (全部を言語モデルに任せない)

転倒・進めない・段差への反応は数十ミリ秒で決まる規則にし (速く、予測可能で、Jetson 級の計算機で動く)、言語モデルは「写真に人はいるか」「人に聞く文面」という急がない判断だけに使う。災害現場の実用性として、どこで何が判断されたかを後から説明できることを重視した。

3.3 失敗を前提にした回復と自己点検

毎手の観測 (GPS、傾き、足元の高さ、進んだ距離) から状態を判定し、転倒・足踏み・はまり込みのそれぞれに回復動作を用意した。司令役の出来事はすべて時刻つきで記録され、動画の字幕と本資料の数字はその記録から自動生成している。

3.4 講座のサンプル (Genesis の Go2 例題) との違い

出発点は Genesis 付属の `go2_env.py` / `go2_train.py` (平地で前に歩く強化学習の例題) である。本提出で足したのは、(1) 現場の地形・横からの衝撃・旋回指令を含む学習環境と、転倒姿勢から始める起き上がり環境および報酬の再設計、(2) GPS もどき・A* 経路計画・高さスキャナ・転倒検知と復帰・足踏みとはまり込みからの脱出・人への確認と通報を行う司令役 (約 300 行)、(3) 頭のカメラ 8 方向とローカル VLM による認識、(4) 同じ乱数列での蹴り耐性・地形・任務の成績表と、出来事の記録からの字幕生成、である。

4. 結果

4.1 見回り任務 (seed 8 通り、余震あり)

see	結果と所要時間	転倒 (すべて自力復帰) / 発見と通報

d		
1	完了 156 秒	転倒 3 / 発見 1
2	完了 177 秒	3 / 1
3	帰還の手前で 320 秒切れ	1 / 1
4	完了 170 秒	3 / 1
5	完了 122 秒	3 / 1 (遠方でも記録)
6	完了 100 秒	1 / 1
7	完了 219 秒	1 / 1
8	完了 130 秒	1 / 1

表 2: 見回り任務 (seed 8 通り、余震は 40 秒に横から 2.5 m/s)。**完了 7/8、発見と通報 8/8、転倒 16 回はすべて自力で復帰**

同じ試験を改良のたびに回し、完了率は 4/8 (真後ろを向く際の足踏み) → 2/4 (箱の密集地でのはまり込み) → 8/8 だが発見 0/8 (人形が砂利区画の地面に埋没) → 5/8 (同じ人形に何度も近づく) → 7/8 と推移した。残る 1 本も最終地点の手前まで到達しており、時間上限を 60 秒延ばせば完了する見込み。

4.2 部品ごとの検証

- 成長: 0 反復では立ち尽くし、50 反復で転ばないが進めず、100 反復で歩く (学習 27 秒)
- 起き上がり: 4 姿勢すべてから 0.75 秒前後で復帰。現場の地形の上では 1.4 秒
- 訓練場 (砂利・坂・箱・階段) で育った方策は、砂利 5.7 m、箱 5.6 m、ピラミッド階段 5.8 m (12 秒)、階段育ちはそれぞれ 3.2 / 5.1 / 5.1 m

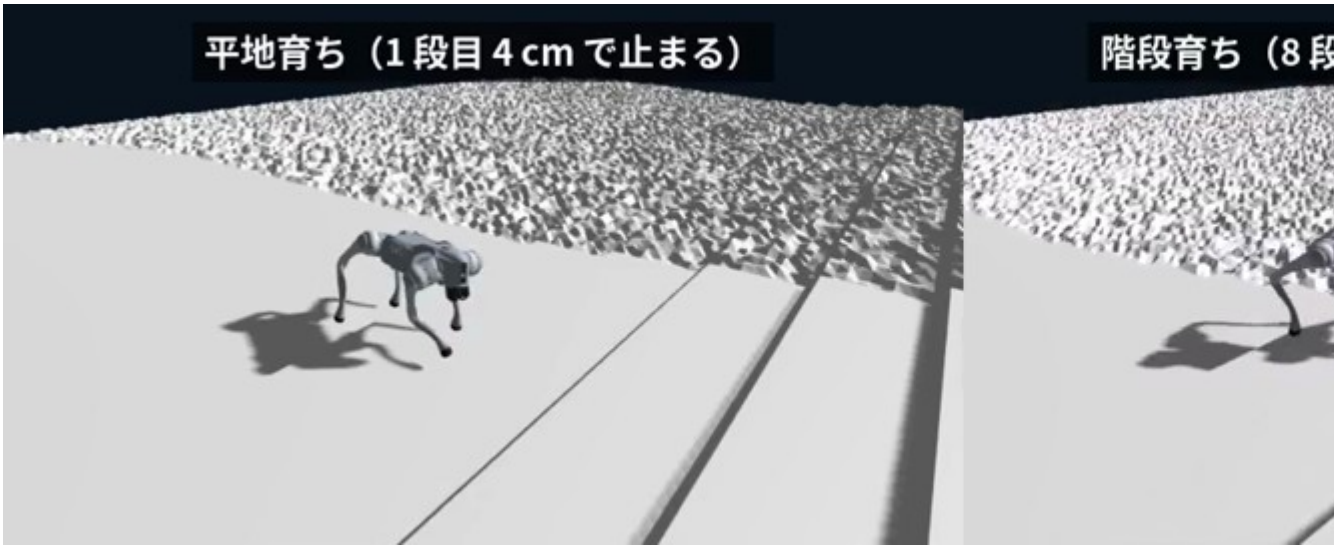


図 2: 同じ階段。左: 平地育ち、右: 階段育ち

5. 失敗と対処

起きたこと	原因	対処
仰向けから起き上がれない	上向き度の報酬を 0 で切っていて、仰向けの間は手がかりが無い	連続値 (仰向け 0 → 横 0.5 → 立ち 1) に
上向き度 0.79 でぴたりと止まる	「立っているときだけ姿勢を罰する」閾値 0.8 の手前で罰を避けた (報酬ハッキング)	姿勢の項は諦め、歩行方策に姿勢を任せる
経由地に向けない	旋回の指令をゼロ固定で学習し、曲がり方を知らない	旋回 ±0.8 rad/s と低速を含めて学習し直す
通報の後 4 分間足踏み	真後ろを向くとき、GPS のばらつきで「右回り/左回り」が数	150° 超なら回る向きを決め、90° を切るまで固定

	秒ごとに裏返る	
壁の前で左右に振れる	経路計画と高さスキャナが同じ壁に別々に反応	スキャナは地図に無い段差だけを見る
人形が見つからない	砂利区画は地面が 0.4 m 高く、人形が埋まっていた / 到着時に瓦礫の山を向いていた	平地に置く / 360° 見回す
学習が落ちる (std < 0)	観測に NaN が混ざる	観測を有限値に丸める

6. 考察と実機への道筋

- 方策は 50 Hz の小さな MLP で、観測の並びと PD ゲインは Unitree の学習環境と同じ。Go2 EDU の低レベル API に薄い変換層で接続でき、司令役と VLM も本体の Jetson 級で動く計算量。GPS は USB の GNSS 受信機で足す
- 実機前に要るのは、摩擦・質量・遅延のランダム化、起き上がり時の自己衝突の確認、カメラの実画像での VLM の検証、通信が切れたときの振る舞い (位置を記録して帰還)
- 学習が 30 秒〜6 分で回るので、仮説 (報酬、地形、外乱) を 1 日に何度も試せた。一番の学びは「失敗を前提に、毎手観測して検証し直すループ」と「練習は本番より厳しく」の 2 つ

付録: 動かし方・コード

GitHub リポジトリの README を参照。主な構成: `mission.py` (司令役)、`planner.py` (A*)、`vlm_check.py` (VLM)、`disaster_site.py` (現場)、`go2_env_*.py` / `go2_train_*.py` (学習環境)、`tools/` (環境構築、字幕)。